

Article

BPMN Assistant: An LLM-Based Approach to Business Process Modeling

Josip Tomo Licardo , Nikola Tanković *  and Darko Etinger 

Faculty of Informatics, Juraj Dobrila University of Pula, Zagrebačka 30, 52100 Pula, Croatia;
josip.tomo.licardo@unipu.hr (J.T.L.); darko.etinge@unipu.hr (D.E.)

* Correspondence: nikola.tankovic@unipu.hr

Featured Application

The proposed system is designed for integration into enterprise Business Process Management (BPM) platforms, with the aim of assisting non-technical business analysts in drafting and modifying executable BPMN models using natural language. This potential to reduce the dependency on technical modeling experts could accelerate the digital transformation workflow.

Abstract

This paper presents BPMN Assistant, a tool that leverages Large Language Models for natural language-based creation and editing of BPMN diagrams. While direct XML generation is common, it is verbose, slow, and prone to syntax errors during complex modifications. We introduce a specialized JSON-based intermediate representation designed to facilitate atomic editing operations through function calling. We evaluate our approach against direct XML manipulation using a suite of state-of-the-art models, including GPT-5.1, Claude 4.5 Sonnet, and DeepSeek V3. Results demonstrate that the JSON-based approach significantly outperforms direct XML in editing tasks, achieving higher or equivalent success rates across all evaluated models. Conformance checking evaluation confirms that generated models preserve executable semantics, with JSON achieving an average F1 score of 0.72 compared to 0.69 for XML, though frontier models like GPT-5.1 and Claude 4.5 Sonnet demonstrated superior precision with direct XML generation. Furthermore, despite requiring more input context, our approach reduces generation latency by approximately 43% and output token count by over 75%, offering a more reliable and responsive solution for interactive process modeling.

Keywords: Business Process Model and Notation (BPMN); Large Language Models (LLMs); process modeling; generative AI; natural language processing



Academic Editor: Bálint Molnár

Received: 24 January 2026

Revised: 21 February 2026

Accepted: 22 February 2026

Published: 25 February 2026

Copyright: © 2026 by the authors.

Licensee MDPI, Basel, Switzerland.

This article is an open access article

distributed under the terms and

conditions of the [Creative Commons](#)

[Attribution \(CC BY\)](#) license.

1. Introduction

Business Process Model and Notation (BPMN) [1] has long served as a standard for modeling business processes, enabling organizations to visualize and optimize their workflows. However, the complexity inherent in creating, editing, and interpreting BPMN diagrams poses significant challenges, particularly for individuals without specialized training [2]. As Recker [3] demonstrates through extensive research, BPMN's over-engineered nature and the prevalent lack of formal training among its users create substantial barriers to effective adoption. This bottleneck often leads to inefficiencies and a reliance on experts, increasing operational costs and delaying decision-making.

A significant challenge in modern organizations is the communication gap between IT departments and business stakeholders. While IT professionals are comfortable with formal modeling notations and technical specifications, business users typically express processes in natural language and informal descriptions. A systematic review by Njanka et al. [4] demonstrates that this disconnect leads to misunderstandings, requirements misalignment, and implementation delays, with communication barriers persisting despite significant investments in alignment initiatives. Furthermore, valuable process knowledge often remains contained in domain experts or scattered across informal documents, making it difficult to capture and formalize [5].

The complexity of extracting process models from unstructured text remains a significant challenge, as highlighted by Bellan et al.'s qualitative analysis [6]. Their research reveals that current methodologies primarily rely on ad-hoc rule-based approaches, which struggle with the complexity of real-world documents. This challenge is further compounded by the limitations of traditional process elicitation methods, as demonstrated by Gonçalves et al. [7], who propose a novel approach integrating group storytelling techniques with text mining to capture the nuances of human-centric activities in process modeling.

Recent research has highlighted the cognitive challenges faced when creating process models, particularly regarding cognitive load and task complexity. Weber et al. [8] introduced a psycho-physiological approach to assess cognitive load during process modeling, using real-time eye-movement analysis to identify task-specific difficulties. Their findings emphasize that process modeling requires substantial cognitive effort, especially in naming activities and managing complex structures. These insights reinforce the need for tools like BPMN Assistant designed to reduce cognitive barriers and potentially enhance accessibility.

The technical barrier to BPMN adoption, combined with the dispersed nature of process documentation, creates a significant impediment to effective process management. Organizations often find themselves in situations where valuable process knowledge exists but remains underutilized because of its informal, distributed nature.

Another critical issue is the dynamic nature of business processes in today's rapidly evolving business environment. Organizations need to frequently update and modify their processes to remain competitive and adapt to changing market conditions [9]. However, the formal nature of BPMN and the expertise required to modify process models often create a bottleneck in implementing these changes, leading to a gap between actual business operations and their formal documentation. This challenge is evidenced by Leopold et al.'s analysis of 585 BPMN models from industry, which revealed persistent quality issues related to model complexity, particularly in areas such as splits and joins, message flows, and model decomposition, highlighting the technical expertise required for effective BPMN modeling [10].

The disconnect between formal process models and actual business operations is further exacerbated by the increasing complexity of modern business processes. These processes often span multiple departments, involve numerous stakeholders, and integrate with various systems and external partners. As van der Aalst and Weijters [11] demonstrate in their seminal work on process mining, organizations face significant challenges in accurately capturing and analyzing these complex processes, including issues with hidden tasks, duplicate activities, and non-free-choice constructs. The traditional modeling approaches struggle to address these challenges while maintaining accessibility for all stakeholders.

Prior to the emergence of Large Language Models (LLMs), numerous attempts were made to automate aspects of process modeling and bridge the natural language gap. Early work by Friedrich et al. [12] demonstrated the potential of rule-based approaches for extracting process models from natural language text, though these systems struggled

with complex sentence structures and domain-specific terminology. The integration of natural language processing into process management has faced significant challenges due to semantic ambiguity and limited contextual understanding. Efforts by Leopold et al. [13] to automatically generate natural language descriptions from process models highlighted both the potential and limitations of traditional NLP approaches in process automation. These early automation attempts, while groundbreaking, were often constrained by their reliance on predefined rules and limited ability to handle variations in natural language expressions [14].

Advancements in artificial intelligence, particularly the emergence of LLMs, have opened new avenues for automating the creation and management of business process models [15–17]. These models have demonstrated exceptional capabilities in understanding and generating natural language, making them well-suited for bridging the gap between textual process descriptions and formal BPMN representations. Recent research by Klievtsova et al. [18] has shown that AI-assisted approaches to process modeling can be particularly effective, with AI-generated models often being preferred over those created by inexperienced human modelers. However, Rebmman et al. [19] highlight that while LLMs show promise in process-related tasks, their effectiveness often depends on proper fine-tuning and task-specific training, particularly for complex semantics-aware operations. This suggests significant potential for LLM-based tools to facilitate access to process modeling capabilities, while also emphasizing the importance of careful implementation and training.

This work presents BPMN Assistant, a system that leverages LLMs to address these challenges. While recent research has demonstrated the potential of AI-assisted process modeling, the critical aspect of model modification and maintenance remains largely unexplored. Our work advances the state of the art by demonstrating the effectiveness of LLMs for editing existing BPMN diagrams via natural language instructions.

Rather than framing BPMN modeling as a one-shot generation problem, this work is conceptualized around the idea of incremental process transformation. We argue that BPMN models should be treated as mutable process structures that can be reliably modified through a constrained set of well-defined operations, instead of being repeatedly regenerated in their entirety. This conceptual perspective motivates the separation of process logic from BPMN 2.0 XML syntax and forms the basis for the structured intermediate representation proposed in this study.

1.1. Research Objectives

This study addresses the following research questions:

- RQ1: How does a structured intermediate representation (JSON) compare to direct standard notation (XML) in terms of generation reliability and editing success rates?
- RQ2: To what extent does a function-based editing approach enable open-weight models to perform complex modeling tasks previously reserved for proprietary frontier models?
- RQ3: How does the trade-off between increased input context and reduced output complexity impact overall system latency and operational efficiency?

1.2. Contribution

We propose a novel approach that utilizes a structured, JSON-based intermediate representation to abstract away the syntactic complexity of BPMN 2.0 XML. By shifting the generative task from producing verbose XML to manipulating a concise JSON structure, our system allows LLMs to focus on process logic rather than formatting rules. At a conceptual level, this approach reframes BPMN editing as a controlled transformation problem, where

modifications are expressed as atomic logical operations rather than as low-level syntactic changes to XML documents.

Our evaluation reveals significant improvements in editing success rates compared to baseline approaches, particularly when using structured representations for model manipulation. Furthermore, we demonstrate that this approach significantly reduces processing latency and output token costs, making it a viable solution for interactive tools.

The implementation of BPMN Assistant is publicly available at <https://github.com/jtlicardo/bpmn-assistant> (accessed on 21 February 2026). The evaluation dataset is publicly available at <https://huggingface.co/datasets/jtlicardo/bpmn-assistant-eval> (accessed on 21 February 2026).

The following sections detail our methodology, system implementation, and evaluation results, providing a foundation for future research in AI-assisted process modeling.

2. Related Work

The integration of LLMs into Business Process Management (BPM) has led to significant developments in process automation and modeling. The introduction of specialized tools like the BPMN-Chatbot [20] and ProMoAI [21] has demonstrated the practical applications of this technology. Initial explorations by Berti and Qafari [22] into using GPT-4 and Bard (Gemini) for process mining tasks showed promising results in interpreting both procedural and declarative models, highlighting the potential of LLMs in the BPM domain. These developments have led to comprehensive frameworks for process modeling with LLMs [23], showing promising results in both automation and model quality.

2.1. Process Extraction and Prompting Strategies

In the area of process extraction, various approaches have demonstrated different methodologies and results. Ferreira et al. [24] developed a semi-automatic method for identifying business process elements from natural language texts, achieving 91.92% accuracy in their prototype implementation through carefully defined mapping rules. Taking a different approach, Neuberger et al. [25] introduced a universal prompting strategy that leverages LLMs for process model information extraction, demonstrating performance improvements of up to 8% F1 score over traditional machine learning methods.

The integration of prompt engineering in BPM has emerged as a promising direction, as discussed by Busch et al. [26]. Their research highlights how prompt engineering can effectively utilize pre-trained language models without extensive fine-tuning, addressing common challenges in process extraction and predictive monitoring. This approach is particularly valuable in scenarios with limited data availability, offering a computationally sustainable solution for BPM applications.

2.2. Interactive Modeling Tools

ProMoAI, introduced by Kourani et al. [21], represents a significant development in LLM-driven process modeling. The tool prompts an LLM to generate constrained Python code for constructing an intermediate POWL representation and incorporates prompt-engineering and iterative error-handling mechanisms to improve reliability. The resulting models are then transformed into standard notations and can be viewed and exported as BPMN and Petri nets (PNML).

The framework underpinning ProMoAI was evaluated in a subsequent benchmark study [27], which compared multiple LLMs using conformance checking (harmonic mean of fitness and precision against ground-truth event logs) as a model-quality measure. In this evaluation, Claude 3.5 Sonnet achieved the highest average quality score (0.93), approaching the ground-truth baseline (0.98).

Köpke and Safan [20] introduced the BPMN-Chatbot, a publicly available web-based tool designed for efficient LLM-based process modeling. The BPMN-Chatbot allows users to generate BPMN process models interactively using text or voice input. It achieved notable efficiency gains, reducing token usage by up to 94% compared to alternative tools like ProMoAI while maintaining a correctness rate of 95%, which surpassed the best competitor's 86%. The authors emphasized the importance of user testing to evaluate the feedback loop's capabilities, which are critical for interactive process design.

While BPMN-Chatbot demonstrates notable strengths, particularly in token efficiency and correctness rates, its empirical evaluation does not isolate the robustness of iterative editing in the feedback loop (e.g., edit success rate over multi-step refinements, structural validity guarantees, or minimal-change behavior), beyond a preliminary technology acceptance test.

Hörner et al. [28] introduce BPMNGen, an LLM-based conversational framework for generating BPMN 2.0 process models from natural-language descriptions and iteratively refining them through user prompts. Their work is distinguished by a comprehensive human-centered evaluation, including controlled user studies assessing semantic alignment, cognitive load, acceptability, and comprehension performance of LLM-generated models compared to expert-created ones. The results indicate that LLM-generated models can achieve expert-level semantic quality for simple and moderately complex processes, while expert intervention remains beneficial for highly complex scenarios.

In contrast, BPMN Assistant focuses on a different aspect of the problem: robust and fine-grained interactive BPMN editing, emphasizing structural correctness, editing reliability, latency, and token efficiency across multiple LLM backends. Rather than evaluating model comprehensibility, our work addresses the engineering challenges of incremental model manipulation and validation, which are complementary to the human-centered quality dimensions studied in BPMNGen.

2.3. Automated Generation and Evaluation Frameworks

In a comprehensive evaluation of process extraction approaches, Bellan et al. [29] analyzed ten state-of-the-art methods for extracting process models from textual descriptions. Their systematic comparison revealed significant variations in performance and methodology among existing tools, with no single approach achieving superior results across all evaluation metrics. This study highlighted the need for standardized evaluation frameworks and emphasized the challenges in automated process extraction that our work aims to address.

Kourani et al. [23] proposed a comprehensive framework leveraging LLMs to automate the generation and refinement of process models from textual descriptions. This framework demonstrated its ability to streamline process modeling tasks while producing sound, executable model outputs. The superiority of this approach was evident in its comparison with traditional methods, particularly in resolving errors and integrating user feedback effectively. GPT-4 showcased strong performance in generating process models, addressing errors, and adapting to user feedback, whereas Gemini struggled with similar tasks.

Nivon et al. [30] introduced a novel approach to automating BPMN process generation from textual requirements, addressing the challenges faced by non-expert users in process modeling. Their methodology employs a three-step approach using a fine-tuned GPT-3.5 model: first, extracting tasks and ordering constraints from textual descriptions; second, constructing an abstract syntax tree (AST) to represent task relationships; and finally, converting the AST into a BPMN process. Their Java-based implementation achieved 78.5% accuracy rate on valid BPMN processes across 200 descriptions, demonstrating the

viability of automated BPMN generation while highlighting areas for improvement with more advanced language models.

These studies collectively illustrate the transformative potential of LLMs in business process modeling. Tools like BPMN-Chatbot, ProMoAI, and Nivon et al.'s automated BPMN generator demonstrate the feasibility of combining generative AI with domain-specific methodologies to achieve higher efficiency, accuracy, and accessibility. Moreover, the benchmarks and frameworks established by researchers provide a foundation for future advancements, emphasizing the importance of careful prompt design, user feedback integration, and domain-specific optimization to realize the full potential of LLMs in BPM. A high-level comparison of the main research objectives and evaluation emphases across these interactive tools is summarized in Table 1.

Table 1. Comparison of interactive BPMN modeling tools by research objective and evaluation emphasis.

Axis	ProMoAI [21,27]	BPMN-Chatbot [20]	BPMNGen [28]	BPMN Assistant
Primary goal	NL → model generation + refinement/optimization	efficient NL → BPMN	NL → BPMN + human-centered quality	reliable NL → BPMN + edit robustness
Output notation	BPMN + PNML	BPMN	BPMN	BPMN
Evaluation focus	conformance-based model quality	correctness + token efficiency + acceptance	semantic alignment + cognitive load + acceptability + comprehension	GED/RGED structural fidelity + failure rate
Evaluation method	automated	automated + user study	user study	automated

3. System Architecture

The BPMN Assistant system architecture, as illustrated in Figure 1, is designed to facilitate seamless interaction between users and BPMN models through natural language inputs. The system comprises three primary components: the Python-based backend, the BPMN layout server, and the Vue.js frontend. Each component plays a critical role in ensuring the system's functionality, efficiency, and usability.

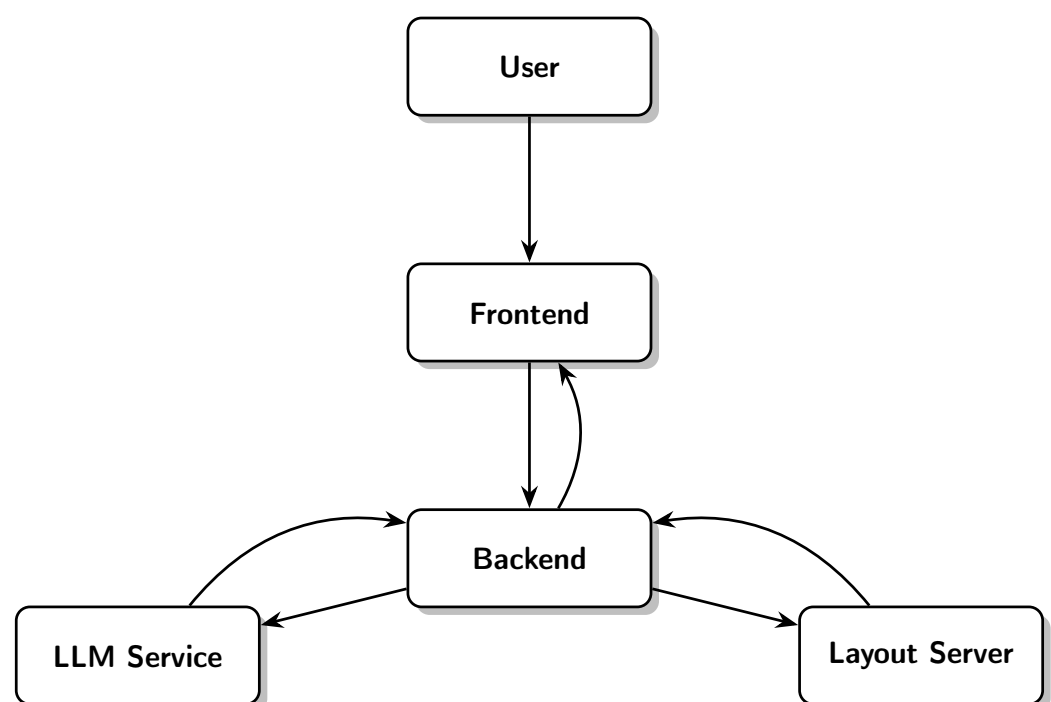


Figure 1. High-level overview of the BPMN Assistant system architecture.

3.1. Backend

The backend, implemented in Python v3.12, serves as the system's core computational engine. Its primary responsibilities include handling user inputs, interacting with the LLM, managing BPMN diagrams, and providing APIs for the frontend. The backend uses the FastAPI framework, chosen for its performance and ease of developing RESTful APIs.

The backend serves as the first point of contact for user inputs, analyzing these inputs to determine the user's intent. This can include conversational queries, such as asking questions about BPMN concepts, or operational commands, such as creating or modifying BPMN diagrams. When a conversational intent is recognized, the backend communicates with the LLM to generate a natural language response. If the input requires an operational response, the backend generates a prompt instructing the LLM to generate or modify a BPMN diagram. These diagrams are initially represented in JSON format and subsequently converted to BPMN XML for compatibility with visualization tools. Additionally, the backend supports the uploading of BPMN files that conform to the supported BPMN subset, enabling users to query or modify existing diagrams within the constraints of the system's JSON representation. The backend includes robust error handling and validation to ensure LLM outputs are accurate and reliable. If invalid JSON is generated, the backend retries the process or notifies the user of the error. By integrating with the BPMN layout server, the backend also ensures that diagrams are enriched with graphical information for enhanced visualization. The backend request-handling flow is summarized in Listing 1.

Listing 1. Pseudocode of the backend request-handling flow.

```

Algorithm: Backend request handling
Input: user text u, optional BPMN file f
Output: text response OR BPMN response

parse_and_validate(u, f)
m := null
if f exists:
    validate_supported_subset(f)
    m := xml_to_json(f)

if is_conversational(u):
    r := llm(build_conversational_prompt(u, m))
    return text_response(r)

repeat up to K times:
    y := llm(build_diagram_prompt(u, m))
    if is_valid_json(y) and conforms_to_schema(y):
        b := apply_layout(json_to_xml(y))
        return diagram_response(b)

return error_response("Invalid LLM output")

```

3.1.1. Model Integration and Selection

Our system supports a wide range of LLMs from different providers, each chosen for its specific strengths and use cases. Table 2 provides an overview of the supported models and their capabilities.

3.1.2. Supported BPMN Elements

The backend facilitates operations on a wide range of BPMN elements to support diverse modeling scenarios. In the category of tasks, the system supports generic tasks, user tasks, service tasks, send tasks, receive tasks, business rule tasks, manual tasks, and

script tasks. To manage process flow logic, the system supports exclusive, parallel, and inclusive gateways.

Table 2. Overview of Integrated AI Models.

Provider	Model	Description
OpenAI	GPT-5.1	OpenAI’s flagship model for coding and agentic tasks.
	GPT-5 mini	Faster, more cost-efficient version of GPT-5, used for well-defined tasks.
	GPT-4o	Earlier-generation multimodal model with broad cross-modal capabilities [31].
Anthropic	Claude 3.5 Sonnet	Earlier-generation model with strong reasoning and long-context capabilities.
	Claude 4.5 Sonnet	Frontier model optimized for agentic reasoning and complex coding tasks.
Google	Gemini 2.0 Flash	Multimodal model with with low latency and a 1M context window [32].
Fireworks AI	Llama 3.3 70B	Meta’s open-source text-only LLM [33].
	Qwen 2.5 72B	Alibaba’s open-source model with strong multilingual capabilities [34].
	Deepseek V3	Open-source model specialized in technical and scientific reasoning [35].

The system also supports events, distinguishing between start, end, and intermediate types. Specifically, it supports generic, timer, and message start events; generic and message end events; as well as generic and message intermediate throw events, and generic, timer, and message intermediate catch events. This selection of elements enables the modeling of specific executable processes and event-driven workflows.

3.2. BPMN Layout Server

The BPMN layout server, implemented using Node.js with the Express.js framework, is dedicated to augmenting BPMN diagrams with graphical information. This server uses the bpmn-auto-layout (<https://github.com/bpmn-io/bpmn-auto-layout>, accessed on 21 February 2026) npm library to generate DI (Diagram Interchange) information, including the graphical coordinates of BPMN elements.

The layout server takes BPMN XML files from the backend and enhances them by adding graphical coordinates, which are essential for visual representation. Although the server efficiently enriches diagrams for most scenarios, it is currently limited in its ability to process multi-pool or multi-lane diagrams due to constraints in the bpmn-auto-layout library. In practice, this means that collaboration diagrams cannot be automatically laid out and reliably visualized inside our web UI; users would need to finalize the layout in downstream BPMN tooling or model participants separately.

This limitation is primarily related to diagram layout and visualization (DI generation), rather than the core JSON-based editing workflow and validation layer. The server is designed to work seamlessly with the backend, exchanging data through REST APIs to ensure smooth integration.

3.3. Frontend

The Vue.js-based frontend provides an intuitive graphical user interface (GUI) for user interaction. Its design focuses on accessibility, enabling non-experts to engage with BPMN modeling tasks effectively.

Figure 2 shows the dual-panel UI used for chat-based interaction and BPMN diagram visualization.

The frontend features a dual-panel design that facilitates interaction with both the LLM and BPMN diagrams. On the left, a chat interface lets users submit natural language queries and view responses in real time. Users can also select their preferred LLM for processing queries. On the right, a BPMN canvas, powered by bpmn.io (bpmn-js), displays the generated or modified BPMN diagrams. This canvas enables users to interact with the

diagrams in a manner similar to traditional desktop tools. The frontend also provides users with feedback during diagram generation or modification, displaying status messages that indicate progress. Once a BPMN diagram is complete, users can download it for offline use or integration with other tools.

The frontend bridges the gap between users and the underlying system, providing a user-friendly interface for BPMN modeling and interaction.

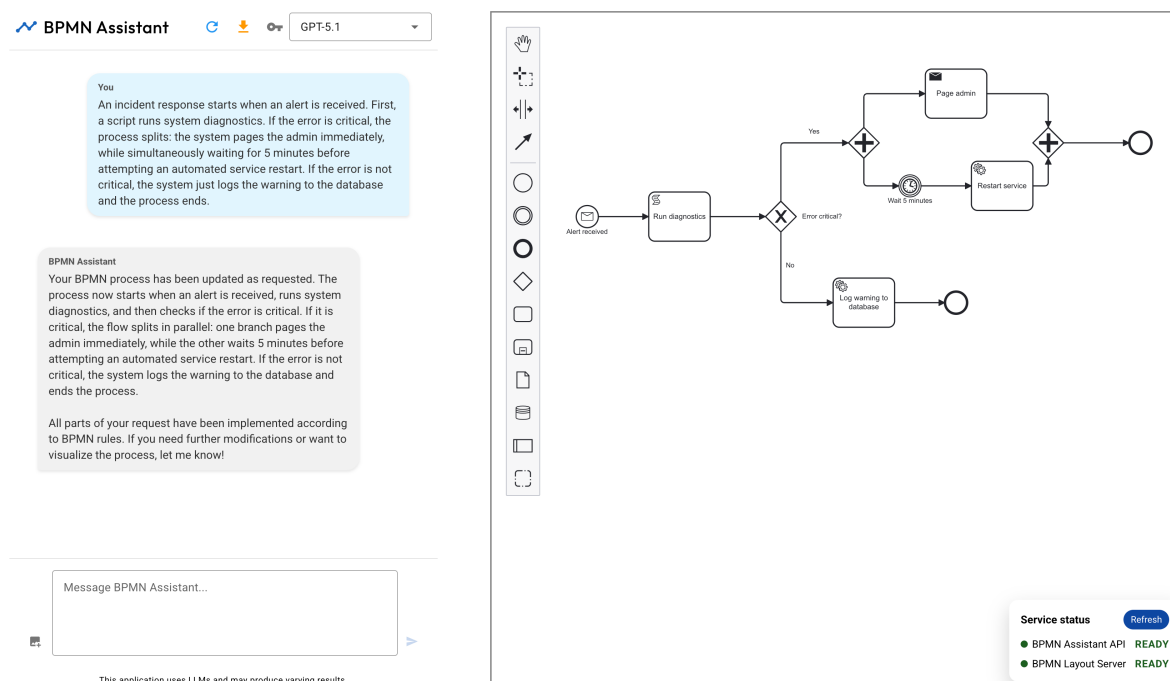


Figure 2. The web application interface features a dual-panel design: a chat interface on the left and a BPMN canvas on the right.

3.4. Data Flow

The system processes data through a structured sequence of interactions, as illustrated in Figure 3.

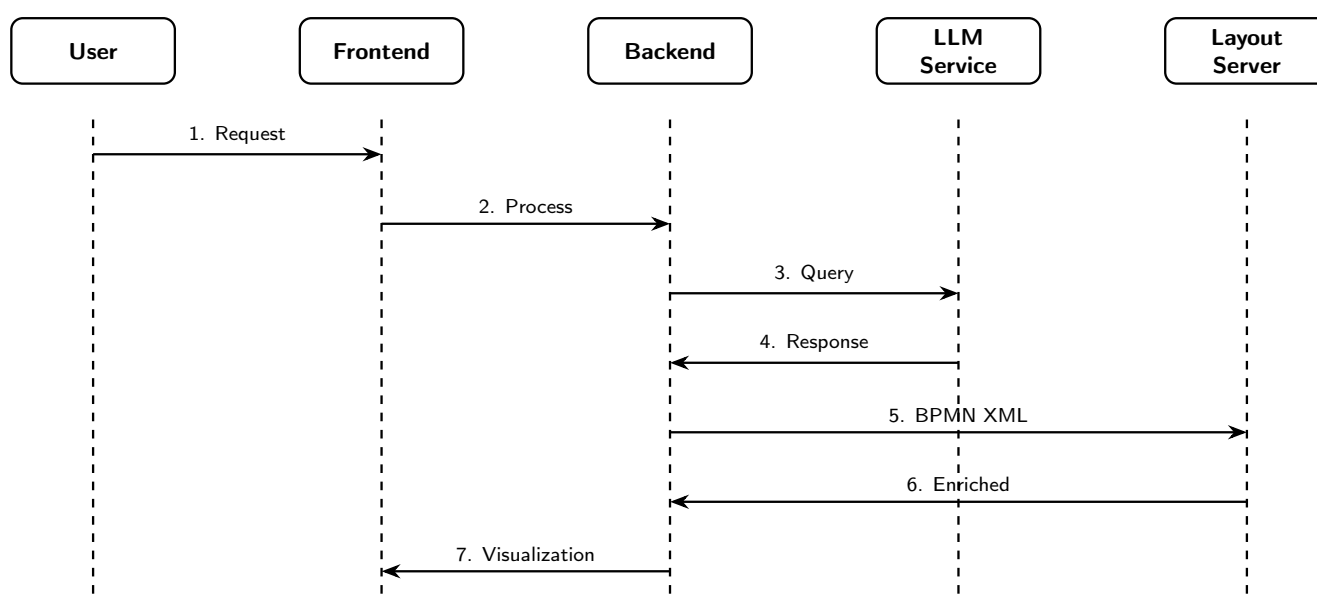


Figure 3. Request sequence flow showing the temporal order of interactions.

The process begins when a user submits a natural language request (Step 1). The frontend forwards this to the backend (Step 2), which performs intent recognition. If a modeling task is identified, the backend queries the LLM Service (Step 3). Unlike standard chatbots, the LLM is instructed to return structured JSON data representing the process or specific editing commands (Step 4). The backend then converts this JSON into standard BPMN 2.0 XML and transmits it to the Layout Server (Step 5). The layout server calculates the X/Y coordinates for the diagram elements and returns the enriched XML (Step 6). Crucially, this process is topology-preserving: the layout server strictly appends Diagram Interchange (DI) metadata without modifying the semantic structure or element identifiers generated by the backend. This ensures that the logical definitions derived from the JSON representation remain consistent throughout the visualization pipeline. Finally, the backend delivers the renderable XML to the frontend for visualization (Step 7).

3.5. JSON Representation of BPMN Diagrams

BPMN Assistant utilizes a hierarchical JSON representation to abstract the verbose XML syntax of standard BPMN 2.0. This structure is designed to be easily generated by LLMs while maintaining sufficient fidelity to map back to valid XML. The representation uses a sequence of elements to describe the process flow. Unless branching logic is introduced via gateways, elements in the process array are executed sequentially.

3.5.1. Tasks

Tasks represent atomic units of work. The system supports a comprehensive set of task types to handle various modeling scenarios. Beyond the generic task, the system supports: `userTask`, `serviceTask`, `sendTask`, `receiveTask`, `businessRuleTask`, `manualTask`, and `scriptTask`. An example of the corresponding JSON structure is shown in Listing 2.

Listing 2. JSON representation of a User Task.

```
{
  "type": "userTask", // or serviceTask, sendTask, etc.
  "id": "task_123",
  "label": "Approve request"
}
```

3.5.2. Events

The system distinguishes between start, end, and intermediate events. Crucially, it supports `eventDefinition` attributes to define specific triggers, such as timers or messages. This allows for the modeling of event-driven architectures. Listing 3 provides an example of an intermediate event with an explicit event definition.

Listing 3. JSON representation of an Intermediate Catch Event.

```
{
  "type": "intermediateCatchEvent",
  "id": "event_timer",
  "label": "Wait 24 hours",
  "eventDefinition": "timerEventDefinition"
}
```

3.5.3. Gateways

Gateways manage flow divergence and convergence. The system supports exclusive (XOR), inclusive (OR), and parallel gateways.

A unique feature of our representation is the `has_join` boolean attribute. Since the JSON structure is hierarchical (nested), `has_join:true` explicitly signals the XML transformer that the branches merge back into a single flow after execution, triggering the generation of a converging gateway node. If `false`, the branches may end independently or loop back to previous elements.

To support cyclic flows (loops) within a nested structure, branches utilize an optional `next` field. This field contains the ID of a target element, allowing the flow to jump to any existing node in the process, breaking the strict hierarchy when necessary. A gateway example illustrating `has_join` and `next` is shown in Listing 4.

Listing 4. JSON representation of an Exclusive Gateway.

```
{
  "type": "exclusiveGateway", // or "inclusiveGateway"
  "id": "gateway_1",
  "label": "Is approved?",
  "has_join": true,
  "branches": [
    {
      "condition": "Yes",
      "path": [ ... ] // Nested sequence of elements
    },
    {
      "condition": "No",
      "path": [],
      "next": "task_start" // Example of a loop-back
    }
  ]
}
```

For inclusive gateways, an `is_default` boolean field is available to designate the default flow path.

Parallel gateways are represented as an array of arrays, where each sub-array constitutes a concurrent execution path. Synchronization is handled implicitly: if the parent flow continues after the gateway, a converging parallel gateway is automatically generated in the XML output. An illustrative parallel gateway structure is provided in Listing 5.

Listing 5. JSON representation of a Parallel Gateway.

```
{
  "type": "parallelGateway",
  "id": "parallel_1",
  "branches": [
    [ { "type": "task", "label": "Path A" } ],
    [ { "type": "task", "label": "Path B" } ]
  ]
}
```

This structured JSON representation serves as the backbone of the BPMN Assistant, allowing for the accurate and efficient translation of user inputs into process models.

For a more complete example of the BPMN JSON representation, please refer to the Appendix A.

3.6. Process Editing Functions

The system supports a set of specialized functions for modifying BPMN diagrams. These functions, outlined in Table 3, enable precise control over process elements while maintaining the structural integrity of the diagram. When the LLM receives an editing request, it analyzes the natural language input and determines which function(s) to call to achieve the desired modifications.

The LLM processes editing requests through a structured approach by first analyzing the user's natural language request to understand the desired changes. It then identifies the affected elements in the current process and determines the appropriate editing functions required to achieve the requested modification. Once identified, the model generates the function calls with the correct parameters and ensures that the proposed changes maintain process integrity through validation.

Table 3. Summary of supported process editing functions.

Function	Parameters	Description
delete_element	element_id	Removes a specified element from the process
redirect_branch	branch_condition, next_id	Redirects a gateway branch to a new target element
add_element	element, before_id *, after_id *	Adds a new element at a specified position
move_element	element_id, before_id *, after_id *	Relocates an existing element within the process
update_element	new_element	Updates properties of an existing element

* Optional parameters, only one should be provided.

Each editing function is designed to perform a specific type of modification while preserving the process's logical flow. For example, when deleting an element, the system automatically handles the reconnection of surrounding elements to maintain process continuity. Similarly, when adding new elements, the system ensures proper integration with existing process flows.

Complex transformations are handled by composing these atomic functions. For instance, modifying an event's trigger definition is executed via a single update_element call. Structural refactoring often involves a sequence of operations; for example, converting a flow might require deleting existing sequence logic and inserting a new gateway structure containing the reorganized tasks via add_element.

The granular nature of these functions allows the LLM to decompose complex editing requests into a series of atomic operations. This approach enhances reliability and makes it easier to validate and verify changes before they are applied to the process model.

3.7. Validation and Soundness

To ensure the generation of syntactically sound BPMN models, the system incorporates a strict validation layer implemented in Python that intercepts the LLM output prior to XML conversion. This validator enforces core BPMN structural constraints, including the uniqueness of element identifiers, the correctness of connectivity between flow elements, the validity of gateway branch hierarchies, and the requirement that each process contains exactly one start event. When a violation is detected, the system initiates a self-correction loop in which the validation error is fed back to the LLM, prompting it to revise the intermediate representation. By acting as a programmatic guardrail, this validation mechanism ensures that the final BPMN XML is both syntactically valid and reliably interpretable by standard BPMN engines. The validation and self-correction loop is summarized in Listing 6.

Listing 6. Pseudocode of the validation and self-correction loop.

Algorithm: Validate-and-correct loop

Input: JSON candidate y Output: validated JSON y^* OR failure

```

repeat up to K times:
    e := validate_bpmn_json(y)
    if e == OK:
        return y

    # Feed back a concise error and request a corrected JSON
    y := llm(build_correction_prompt(y, e))

return failure("Could not produce valid JSON")

```

4. Evaluation

The evaluation of BPMN Assistant focuses on assessing its accuracy through Graph Edit Distance (GED) and Relative Graph Edit Distance (RGED), two widely recognized metrics for measuring process model similarity [36–38]. Traditional GED methods often struggle with gateway semantics and execution probabilities, limiting their effectiveness in BPMN similarity measurements [39]. Schoknecht et al. [40] provide a comprehensive review of process model similarity techniques, emphasizing that hybrid approaches—integrating syntactic, semantic, and behavioral comparisons—tend to produce more reliable results. However, many of these methods require domain-specific adaptations to be effective for BPMN.

Our evaluation methodology evaluates both the structural fidelity of the generated models (via Graph Edit Distance metrics) and their execution semantics (via conformance checking on simulated event logs). However, to clearly define the scope of this study, it is important to note what is excluded: we do not assess human-in-the-loop usability, cognitive load, or visual layout aesthetics. Furthermore, our semantic evaluation relies on simulated traces rather than real-world production logs, and we do not conduct execution performance benchmarking. These aspects represent valuable dimensions for assessing process modeling tools but fall outside the scope of the current research, which primarily examines the efficacy of different intermediate representations.

4.1. Prompting Protocol and Templates

To support replicability, we provide the exact prompt template used to instruct models in the XML baseline (Appendix B). For the JSON-based approach, BPMN Assistant uses multiple prompt templates (e.g., initial generation, atomic edit-function calling, and error-correction after validation failures); these templates are available in the public BPMN Assistant repository.

All models were evaluated using their default provider-side decoding parameters (e.g., temperature, top- p) to reflect standard API behavior. No model-specific hyperparameter tuning was performed, ensuring that the reported results represent baseline capabilities accessible to general developers rather than optimized or hand-tuned configurations.

In the XML baseline, we relied entirely on provider defaults and did not explicitly set temperature, top- p , or maximum output tokens. In contrast, the JSON-based pipeline is orchestrated by BPMN Assistant itself, where decoding behavior may vary across turns and models due to iterative function-calling, error-correction, and control logic. As a result, there is no single fixed temperature or decoding configuration for JSON runs. The reported results should therefore be interpreted as end-to-end pipeline baselines rather than a strict apples-to-apples comparison under identical decoding controls.

4.2. Dataset Composition

To ensure a robust evaluation across diverse scenarios, we constructed a dataset of 60 process descriptions spanning 20 distinct business domains (e.g., Marketing, Healthcare, Logistics). The dataset was curated to evaluate the system’s ability to handle fundamental BPMN control-flow constructs, including sequential tasks, exclusive decision points (XOR-split/join), and concurrent execution paths (AND-split/join). This focus on core structural patterns provides a baseline for assessing the generation reliability of the underlying JSON representation.

The evaluation set was generated using a fixed prompt template and the OpenAI gpt-4.1 model. Each description was constrained to 7–8 activities and formulated in natural language while explicitly discouraging BPMN-specific terminology. Three descriptions were generated for each of the 20 business domains, after which the outputs were manually screened to remove ambiguous or degenerate cases.

Ground-truth BPMN diagrams were then created from these descriptions by multiple BPMN-trained annotators, including the authors, following a consistent modeling guideline. All diagrams were subsequently reviewed and corrected by the authors to ensure syntactic validity and semantic alignment with the corresponding textual descriptions.

4.3. Graph Edit Distance (GED) and Relative Graph Edit Distance (RGED)

GED quantifies the cost of transforming one BPMN diagram into another by counting the minimum number of graph edit operations (node insertion, deletion, or substitution) required. To address the semantic variability inherent in natural language generation (e.g., an LLM generating “Process Order” versus “Handle Order”), we implemented a custom two-stage evaluation pipeline (implementation available at: <https://github.com/jtlicardo/bpmn-ged>, accessed on 21 February 2026).

First, we perform semantic label normalization. Raw BPMN XML files are parsed into a graph structure, and a lightweight LLM (GPT-5 mini, see Appendix B.3 for prompt and settings) is employed to map semantically equivalent labels to identical abstract tokens (e.g., mapping both “Submit Order” and “Send Order” to token “A”). This ensures that the metric evaluates logical flow accuracy rather than penalizing surface-level lexical differences.

Because this normalization step is model-driven, it can introduce minor measurement noise if two unrelated labels are mapped to the same token (over-merging) or if two equivalent labels are mapped to different tokens (under-merging). We performed a manual spot-check of the produced mappings on a subset of the evaluation pairs to confirm that the normalization aligns with human judgment; overall, the mappings were consistent with the intended semantics of the tasks and flows.

Second, we compute the GED using the NetworkX library with a specific cost configuration designed to distinguish between minor syntactic errors and major structural hallucinations. The cost functions are defined as follows:

Insertion and Deletion	The cost for inserting or deleting a node or edge is set to 1.0.
Node Substitution	The substitution cost $c_{sub}(n_1, n_2)$ between two nodes is determined by:

$$c_{sub}(n_1, n_2) = \begin{cases} 0.0 & \text{if } \text{label}(n_1) = \text{label}(n_2) \wedge \text{type}(n_1) = \text{type}(n_2) \\ 0.5 & \text{if } \text{label}(n_1) = \text{label}(n_2) \wedge \text{type}(n_1) \neq \text{type}(n_2) \\ 1.0 & \text{otherwise} \end{cases} \quad (1)$$

Edge Matching	Edges are matched based on the equality of their normalized labels.
----------------------	---

The partial penalty of 0.5 for type mismatches allows us to distinguish cases where the model correctly inferred the intent (the label) but selected the wrong BPMN element type (e.g., using a generic *Task* instead of a *User Task*) from cases where the model hallucinated an entirely incorrect step.

RGED normalizes the GED by considering the complexity of the involved graphs, providing a metric that is independent of diagram size:

$$\text{Relative GED} = \frac{\text{GED}(G_1, G_2)}{\text{GED}(G_1, \emptyset) + \text{GED}(G_2, \emptyset)} \quad (2)$$

$$\text{Similarity} = 1 - \text{Relative GED} \quad (3)$$

where $\emptyset$ represents an empty graph. This results in a similarity score where 1.0 represents a semantically identical structure and 0.0 represents total dissimilarity.

To implement the calculation, we utilized the NetworkX Python library, which provides efficient algorithms for computing graph edit distance. Unlike prior approaches that rely on domain-specific heuristics or ML-based enhancements [38], our implementation applies standard GED operations to measure similarity between BPMN diagrams, albeit with the pre-processing step of semantic normalization.

By minimizing the edit distance to human-generated ground truth models, we utilize GED and RGED as proxies for structural fidelity and logical correctness. However, it is important to note that these metrics do not explicitly capture higher-level qualitative factors such as visual readability, layout simplicity, or cognitive load, which are emphasized in studies like those by Huang and Kumar [41] and Pavlicek et al. [42].

4.4. Semantic Evaluation via Conformance Checking

To complement the structural similarity metrics and address the gap between syntactic correctness and executable semantics, we incorporated a behavioral evaluation using conformance checking from process mining. This approach validates whether generated models exhibit execution behavior equivalent to their ground-truth counterparts, which is critical for executable BPMN models intended for workflow automation.

For each generated BPMN model in our evaluation dataset, we performed semantic evaluation using PM4Py [43]. Both the ground-truth and generated BPMN models were converted into Petri nets, the standard formal representation for process execution semantics. From each ground-truth Petri net, we simulated an event log containing 200 execution traces, with each trace containing up to 50 activities, representing behavioral scenarios that the ground-truth model permits. We then replayed this ground-truth event log on the corresponding generated Petri net using token-based replay, computing three conformance metrics: fitness, precision, and F1 score.

Fitness measures how well the generated model can reproduce the ground-truth behavior without deviations, with values ranging from 0 to 1 where higher is better. High fitness indicates that most behavioral traces from the ground-truth log can be executed by the generated model without missing tokens. Precision quantifies the extent to which the generated model avoids allowing unnecessary extra behavior beyond what is needed to explain the log, also ranging from 0 to 1 with higher values indicating better alignment. High precision indicates the model is not overly permissive and avoids spurious execution paths. The F1 score represents the harmonic mean of fitness and precision, providing a single summary metric balancing the ability to reproduce ground-truth behavior with the constraint of not introducing extraneous behavioral possibilities.

This conformance-based evaluation directly addresses the research goal of generating executable process models by verifying that the generated BPMN artifacts preserve the intended execution semantics, not merely structural appearance.

The results of the conformance checking evaluation are presented in Table 4. Both JSON and XML approaches achieved high fitness scores after label normalization, averaging 0.84 and 0.82 respectively, indicating that generated models can successfully reproduce the majority of execution traces from the ground truth. The overall behavioral alignment reveals nuanced patterns across different model capabilities. While JSON achieved a higher average precision of 0.69 compared to XML's 0.66, this advantage was not uniform across all models. Notably, the most capable frontier models exhibited different behavior: GPT-5.1 achieved significantly higher precision with XML (0.71) compared to JSON (0.54), and Claude 4.5 Sonnet similarly performed better with XML (0.75 precision, 0.78 F1) than with JSON (0.73 precision, 0.74 F1). This suggests that these advanced models possess sufficient capacity to handle the syntactic complexity of direct XML manipulation while maintaining tight semantic constraints. In contrast, smaller or open-weight models demonstrated substantial improvements with the JSON approach: DeepSeek V3 achieved 0.76 precision with JSON compared to only 0.59 with XML, Gemini 2.0 Flash showed 0.77 versus 0.63, and similar patterns emerged across Llama, Qwen, and GPT-4o. The JSON representation appears to provide crucial scaffolding for models that would otherwise struggle with the verbosity and structural complexity of raw XML, while frontier models can leverage their enhanced reasoning capabilities to excel with either representation.

Table 4. Semantic conformance evaluation results showing fitness, precision, and F1 scores for both JSON and XML approaches with normalized labels. Higher values indicate better behavioral alignment with ground-truth models.

Model	JSON			XML		
	Fitness	Precision	F1	Fitness	Precision	F1
GPT-5.1	0.79	0.54	0.56	0.81	0.71	0.74
GPT-5 mini	0.83	0.66	0.69	0.81	0.62	0.65
GPT-4o	0.85	0.72	0.75	0.83	0.72	0.74
Claude 4.5 Sonnet	0.84	0.73	0.74	0.86	0.75	0.78
Claude 3.5 Sonnet	0.85	0.66	0.70	0.86	0.62	0.68
Gemini 2.0 Flash	0.86	0.77	0.80	0.81	0.63	0.64
Llama 3.3 70B Instruct	0.83	0.70	0.74	0.82	0.66	0.71
Qwen 2.5 72B Instruct	0.83	0.65	0.70	0.82	0.64	0.67
DeepSeek V3	0.85	0.76	0.79	0.81	0.59	0.62
Average	0.84	0.69	0.72	0.82	0.66	0.69

4.5. Comparison with Baseline XML Editing

To establish the effectiveness of BPMN Assistant, we conducted a comparative analysis between our atomic editing approach and a full-regeneration baseline (implemented here as direct XML generation). This baseline serves as a proxy for conversational refinement workflows used by state-of-the-art tools such as BPMN-Chatbot [20] and ProMoAI [21]. Both systems prompt the LLM to produce a revised structured representation of the process conditioned on the current model state (e.g., intermediate JSON in BPMN-Chatbot, or constrained code/POWL construction in ProMoAI). BPMN-Chatbot explicitly targets token efficiency via its intermediate JSON representation, whereas ProMoAI primarily leverages formal structure and constrained generation to improve reliability.

For the XML baseline, we utilized a system prompt instructing the model to act as a BPMN expert and output valid BPMN 2.0 XML directly, without intermediate steps or function calling. This represents the standard ‘zero-shot’ approach common in current LLM

applications. For this comparison, we selected a representative set of diagram modification tasks and executed them using both representations. We evaluated the approaches using RGED and its derived similarity score, as defined in Equations (2) and (3).

4.6. Generation Accuracy

For the purposes of this evaluation, a “failure” is defined as any generation event that resulted in an unrenderable model. This encompasses both syntactic invalidity—such as unclosed XML tags or malformed JSON objects that prevent parsing—and structural inconsistencies, specifically reference hallucinations where sequence flows target node identifiers that do not exist within the element set. Any output falling into these categories was recorded as a failure in Tables 5 and 6 and treated as a non-functional result.

Calculated similarity scores represent the mean of successful generation attempts only; failed attempts were excluded from the similarity aggregation and are reported separately in the ‘Failures’ column to provide a distinct measure of reliability.

The comparative analysis revealed that our JSON-based representation achieved an average similarity score of 0.72 compared to 0.70 for direct XML generation. While this indicates a slight numerical advantage for JSON, the difference is minimal, suggesting that both representations perform similarly in terms of structural accuracy. However, JSON demonstrated greater reliability, with fewer total failures across models.

Table 5. Evaluation results showing similarity scores for different models using both our JSON-based BPMN representation and direct XML generation approaches. Each model was evaluated on 60 BPMN generation tasks.

Model	JSON	XML	Failures (JSON)	Failures (XML)
GPT-5.1	0.73	0.70	0	0
GPT-5 mini	0.77	0.68	0	0
GPT-4o	0.71	0.68	0	0
Claude 4.5 Sonnet	0.80	0.76	0	0
Claude 3.5 Sonnet	0.72	0.72	0	0
Gemini 2.0 Flash	0.70	0.66	0	0
Llama 3.3 70B Instruct	0.68	0.70	0	4
Qwen 2.5 72B Instruct	0.69	0.72	2	6
DeepSeek V3	0.72	0.69	0	1

Table 6. Average similarity scores and total failures per modality across all generated models.

Modality	Average Score	Total Failures
JSON	0.72	2
XML	0.70	11

Beyond similarity scores, JSON continues to outperform XML in terms of efficiency, a trend that holds even with newer, faster models. Latency measurements represent the total API response time recorded via the respective commercial providers (OpenAI, Anthropic, Google, and Fireworks AI). This metric reflects the real-world performance experienced by end-users of cloud-based LLM services. As shown in Table 7, JSON-based BPMN generation achieved a mean latency of 13.42 s compared to 24.82 s for XML. Figure 4 visualizes the latency comparison between the two representations. Additionally, while JSON requires a richer input prompt (averaging 2678 tokens), it produces significantly more concise outputs (688 tokens vs. 1832 tokens). Figures 5 and 6 visualize this trade-off for input and output tokens, respectively. This reduction in output tokens is particularly advantageous for cost scaling, as output tokens are typically significantly more expensive

than input tokens. Furthermore, the financial overhead of the increased input context is effectively mitigated by prompt caching technologies, which drastically reduce the cost of processing the static schema definitions and instructions required by our approach. To quantify these financial implications, Table 8 details the API pricing assumptions, and Table 9 outlines the mean token usage per modality.

Table 7. Summary of latency and token usage between JSON and XML representations for BPMN generation.

Metric	JSON	XML
Mean Latency (s)	13.42	24.82
Average Input Tokens	2678.63	474.05
Average Output Tokens	688.04	1832.48

The full per-generation cost breakdown (JSON vs. XML; cached vs. uncached) is reported in Appendix C (Table A1). Savings from XML → JSON are summarized in Table 10.

Table 8. Token pricing (per million tokens) as of 9 February 2026.

Provider	Model	Input	Cached Input	Output
OpenAI	GPT-4o	2.50	1.25	10.00
OpenAI	GPT-5.1	1.25	0.125	10.00
OpenAI	GPT-5 mini	0.25	0.025	2.00
Anthropic	Claude 4.5 Sonnet	3.00	0.30	15.00
Anthropic	Claude 3.5 Sonnet	3.00	0.30	15.00
Gemini API	Gemini 2.0 Flash	0.10	0.025	0.40
Fireworks AI	Llama 3.3 70B Instruct	0.90	0.45	0.90
Fireworks AI	Qwen 2.5 72B Instruct	0.90	0.45	0.90
Fireworks AI	DeepSeek V3	0.56	0.28	1.68

Table 9. Mean token usage per generation used in the cost model.

Model	Modality	Input Tokens	Output Tokens
GPT-4o	JSON	2487.85	483.07
GPT-4o	XML	437.68	1348.97
GPT-5.1	JSON	2573.68	568.12
GPT-5.1	XML	449.68	1602.03
GPT-5 mini	JSON	2580.15	888.60
GPT-5 mini	XML	449.68	1619.22
Claude 4.5 Sonnet	JSON	3089.90	617.45
Claude 4.5 Sonnet	XML	520.20	1833.30
Claude 3.5 Sonnet	JSON	2933.20	607.48
Claude 3.5 Sonnet	XML	519.20	1610.55
Gemini 2.0 Flash	JSON	2795.17	556.00
Gemini 2.0 Flash	XML	465.17	1543.10
Llama 3.3 70B Instruct	JSON	2515.56	352.03
Llama 3.3 70B Instruct	XML	485.32	1513.13
Qwen 2.5 72B Instruct	JSON	2619.89	392.41
Qwen 2.5 72B Instruct	XML	468.32	1522.40
DeepSeek V3	JSON	2512.23	394.27
DeepSeek V3	XML	471.23	1470.80

Table 10. Savings from XML → JSON per generation (USD cents) under two assumptions: (i) no caching and (ii) warm cache. Positive values indicate JSON is cheaper.

Model	No Cache (c)	Cached (c)
GPT-4o	0.353357	0.609629
GPT-5.1	0.768410	1.007360
GPT-5 mini	0.092862	0.140798
Claude 4.5 Sonnet	1.052865	1.746684
Claude 3.5 Sonnet	0.780405	1.432185
Gemini 2.0 Flash	0.016184	0.033659
Llama 3.3 70B Instruct	−0.078223	0.013138
Qwen 2.5 72B Instruct	−0.091942	0.004878
DeepSeek V3	0.066561	0.123709

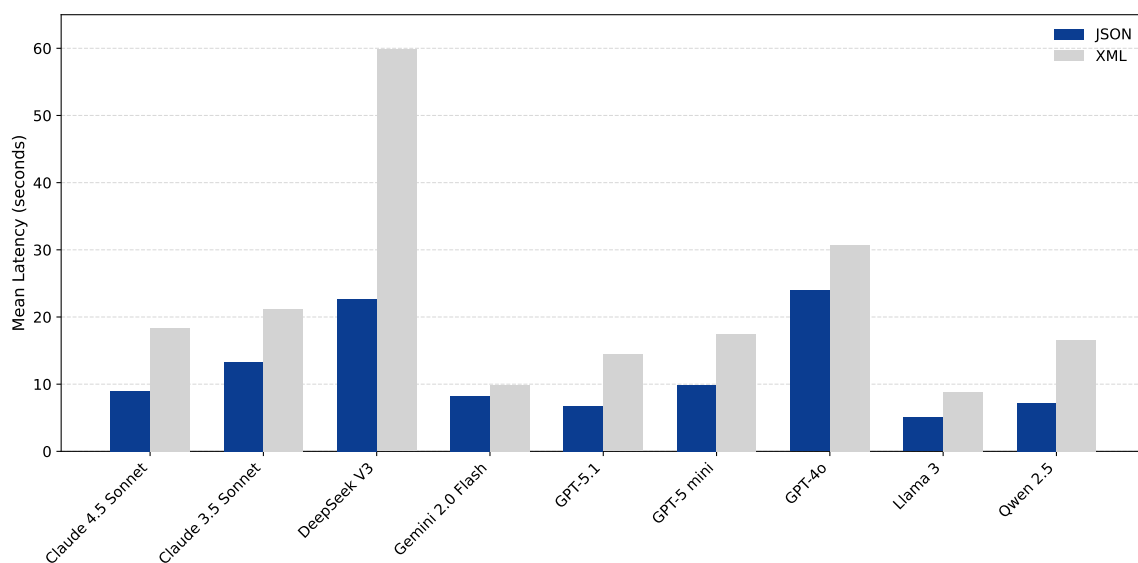


Figure 4. Latency comparison between JSON and XML-based BPMN generation.

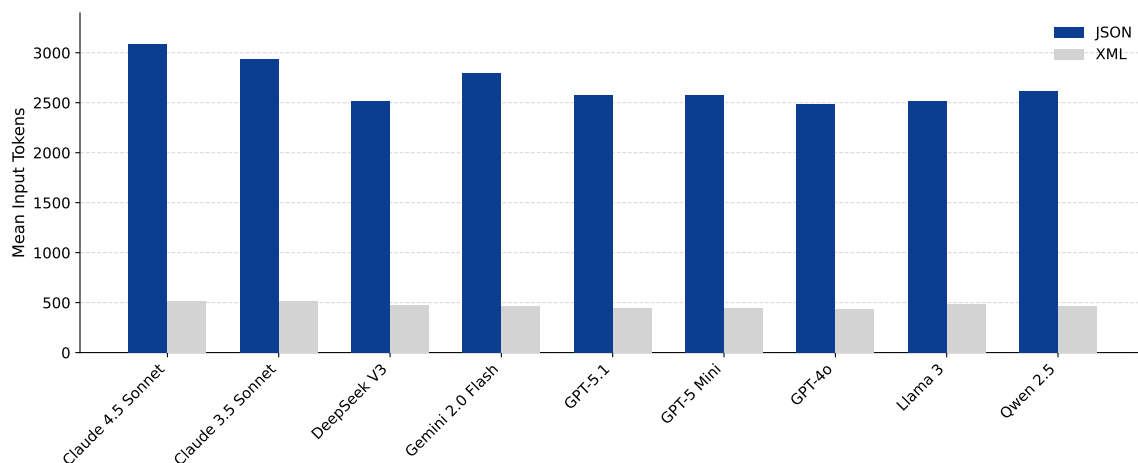


Figure 5. Mean input token usage for JSON vs. XML prompts in BPMN generation.

4.7. Editing Capabilities

In addition to diagram generation, we evaluated the models' ability to interpret natural language editing instructions and perform the requested modifications. For this evaluation phase, we manually curated a dataset of 40 specific modification requests targeting various

process elements. We employed a binary success/fail metric, as editing tasks involve correctly understanding the request and applying appropriate modifications to an existing diagram without corrupting the remaining structure.

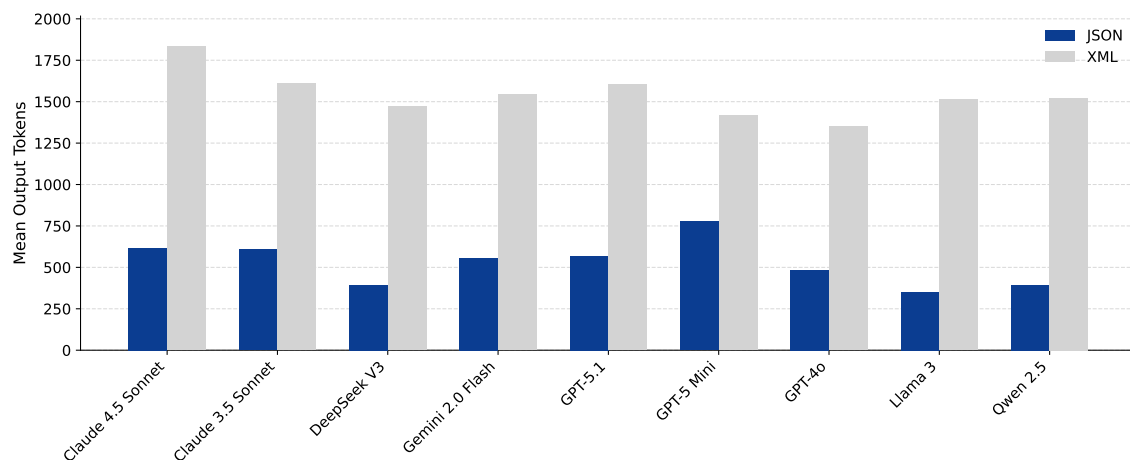


Figure 6. Mean output token usage for JSON vs. XML prompts in BPMN generation.

The editing evaluation set comprises 40 natural language requests covering a diverse range of structural modifications. The tasks include atomic operations, such as element deletion (e.g., “Delete the exclusive gateway”) and insertion (e.g., “Add a task where the system performs checks”), as well as complex structural refactoring, such as converting parallel flows into sequential sequences or introducing new conditional branching logic. The requests target core BPMN constructs, including tasks, gateways, and events. The full dataset is available in the public repository.

To determine validity, we employed a two-stage verification process. First, a Python-based automated validator checked all outputs for syntactic correctness and referential integrity (e.g., ensuring no broken XML tags or references to non-existent IDs). Outputs failing this stage were automatically classified as failures. Second, the syntactically valid outputs underwent expert manual verification to assess semantic correctness. This step involved a binary check against the editing prompt (e.g., verifying that a “delete task” operation actually removed the target node and correctly reconnected the surrounding sequence flows). Given the objective nature of these boolean operations, a single expert evaluator was deemed sufficient.

To minimize subjectivity in the single-rater evaluation, we employed a strict verification rubric based on outcome integrity. A modification was classified as a ‘Success’ only if it satisfied three conditions:

- **Intent Realization:** The requested semantic change (e.g., “delete task”, “sequentialize flow”) was fully implemented in the final graph topology.
- **Structural Validity:** The resulting model remained syntactically valid and fully connected (verified by the automated validator).
- **Invariant Preservation:** No unintended changes occurred to unrelated process segments (zero collateral damage).

For complex requests requiring multiple atomic operations, ‘Success’ was granted only if the entire sequence was executed correctly, effectively treating partial success as a failure.

The results, summarized in Table 11, show that models consistently achieved higher success rates when using our JSON-based approach compared to direct XML manipulation. The performance gap is particularly evident in open-weight models; for instance, DeepSeek

V3 achieved a 50% success rate with JSON but failed almost entirely with XML (8%). This suggests that the structured intermediate representation helps mitigate the complexities of verbose XML syntax, allowing models to focus on the logical changes rather than the syntactic overhead of the BPMN standard. Figure 7 provides a visual comparison of the success rates across models.

Even for flagship models like Claude 4.5 Sonnet, which achieved an 85% success rate in both modalities, the JSON approach offers superior programmatic verifiability. Because the JSON output adheres to a strict internal schema, the system can validate logical consistency (e.g., ensuring all branch targets exist) before conversion. In contrast, validating generated XML often requires complex parsing that may fail on minor syntax hallucinations.

Table 11. Success rates for diagram editing based on natural language instructions, comparing our JSON-based approach with direct XML editing. Each model was evaluated on 40 diverse editing tasks.

Model	JSON	XML
GPT-5.1	0.83	0.75
GPT-5 mini	0.73	0.53
GPT-4o	0.55	0.30
Claude 4.5 Sonnet	0.85	0.85
Claude 3.5 Sonnet	0.68	0.65
Gemini 2.0 Flash	0.45	0.33
Llama 3.3 70B Instruct	0.38	0.30
Qwen 2.5 72B Instruct	0.38	0.25
DeepSeek V3	0.50	0.08

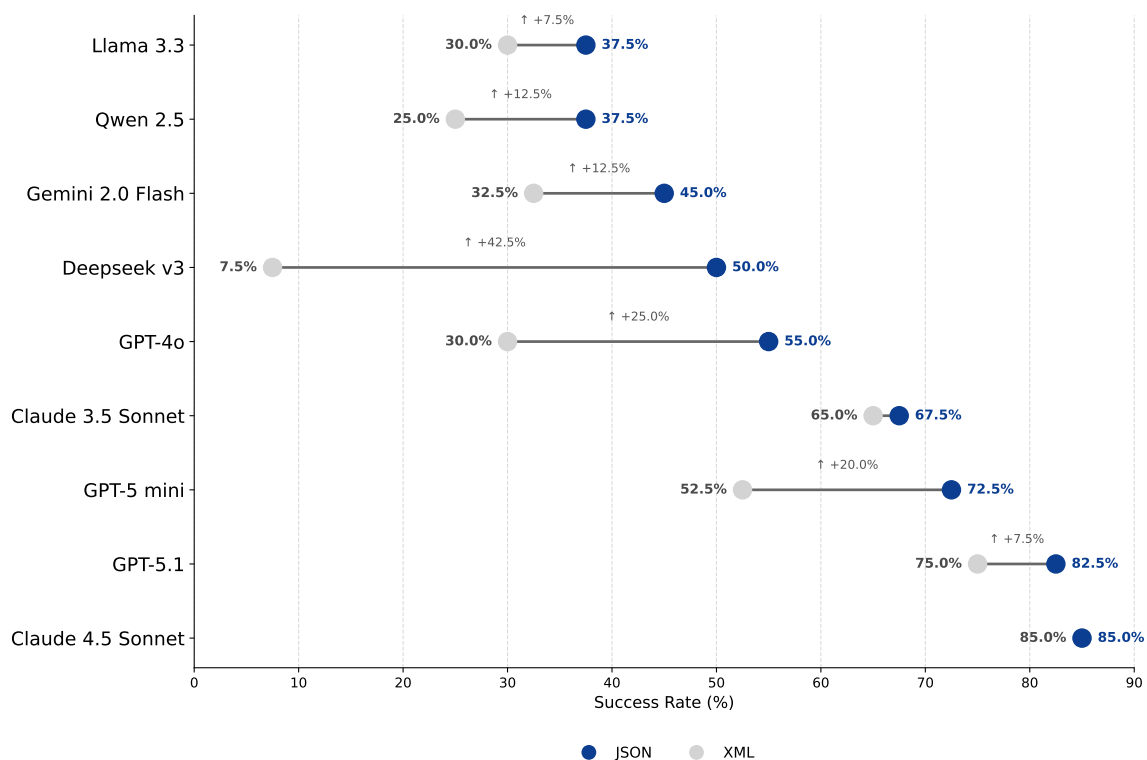


Figure 7. Success rate comparison for diagram editing tasks between JSON and XML-based BPMN representations.

As detailed in Table 12, the trade-off for this reliability is context size. The JSON editing approach requires supplying the full BPMN intermediate representation and various

process examples in the prompt, resulting in an average of 22,071 input tokens compared to 5149 for XML. However, this ‘up-front’ cost pays dividends in speed: the JSON approach reduces generation latency by nearly 43% (20.35 s vs. 35.63 s) and reduces output verbosity by over 75% (607 vs. 2630 tokens). In an interactive tool, this lower latency is critical for user experience.

Table 12. Comparative performance metrics between JSON and XML-based approaches for BPMN editing.

Metric	JSON	XML
Average Latency (s)	20.35	35.63
Average Input Tokens	22,071.44	5149.42
Average Output Tokens	607.92	2630.44

5. Discussion

These findings empirically validate the initial conceptual assumption of this work, namely that constraining LLM interaction through structured representations leads to more reliable and controllable BPMN model manipulation.

5.1. The Efficacy of Structured Representations (RQ1)

Our evaluation reveals a distinct dichotomy between generation and editing performance. In de novo generation tasks, the difference between JSON and XML approaches was marginal (e.g., GPT-5.1 achieved 0.73 with JSON vs. 0.70 with XML). This suggests that for “tabula rasa” tasks, modern LLMs are sufficiently capable of managing XML syntax when generating from scratch.

However, the editing tasks expose the fragility of direct XML manipulation. The JSON-based approach achieved consistently higher success rates across all models. We attribute this difference to the verbosity and structural fragility of direct XML manipulation. When editing XML, an LLM must navigate verbose opening and closing tags, often losing track of the hierarchical structure or hallucinating invalid ID references. In contrast, our JSON schema and the associated atomic editing functions (e.g., `add_element`) force the model to focus on the logical operation rather than the syntactic overhead. This structural constraint effectively acts as a guardrail, strongly promoting that modifications remain valid by design.

5.2. Broadening Access to Process Modeling (RQ2)

Perhaps the most significant finding is the impact of our approach on open-weight models. While frontier proprietary models like Claude 4.5 Sonnet performed admirably in XML editing (85%), open-weight models such as DeepSeek V3 struggled significantly, achieving only an 8% success rate with XML. However, when switching to our JSON-based function calling approach, DeepSeek V3’s success rate jumped to 50%.

This has notable implications for enterprise adoption. Many organizations in regulated industries (finance, healthcare) cannot send sensitive process data to external APIs like OpenAI or Anthropic due to data privacy concerns. Our results demonstrate that by using a structured intermediate representation, organizations can deploy locally hosted, open-weight models to perform complex process modeling tasks that were previously the domain of massive frontier models. This effectively lowers the barrier to entry for secure, on-premise AI process assistants.

5.3. Efficiency and Latency Trade-Offs (RQ3)

The shift to a JSON-based editing workflow introduces a counter-intuitive trade-off: we drastically increase the input context size to decrease latency. As shown in Table 12, the JSON approach requires supplying the entire current state of the process, leading to a $\sim 4\times$ increase in input tokens compared to the XML baseline.

However, in the current landscape of LLM economics and performance, this trade-off is highly favorable. Input tokens are computationally cheaper and faster to process than output tokens. By shifting the complexity to the input (the context) and restricting the output to concise JSON function calls, we achieved a 43% reduction in total latency (20.35 s vs. 35.63 s). For interactive tools where user experience depends on responsiveness, this reduction is critical. Furthermore, the 75% reduction in output tokens leads to significant cost savings at scale, as output tokens typically cost 3–4 times more than input tokens across major providers. This efficiency is further amplified by prompt caching, which allows the static JSON schema definitions—the bulk of our input context—to be processed at a fraction of the standard input cost.

5.4. Practical Case Study and Scope Limitations

To illustrate the practical capabilities and limitations of the system, we analyze a representative case study involving an order processing workflow.

To demonstrate practical utility, consider a realistic order-fulfillment workflow edited across multiple turns. In turn 1, the user asks for “order intake, stock check, payment, and shipping notification,” and the system generates a valid process with a stock decision and payment path. In turn 2, the user refines the exception path: “if out of stock, wait 24 h and retry.” The editor applies iterative function-level updates (e.g., `add_element` to insert an `intermediateCatchEvent` with `timerEventDefinition`, then `redirect_branch` to loop back to the stock-check gateway), validating BPMN after each step. In turn 3, the user requests “for high-value orders, run fraud screening and manager approval in parallel before payment,” which is represented by adding a `parallelGateway` branch structure and rejoining before downstream tasks. This illustrates that the system supports meaningful multi-turn refinement of control-flow logic rather than one-shot generation only.

The same case also exposes enterprise boundaries of the current subset. If the user asks to “assign payment to the Finance lane and hand off to a Supplier pool,” those requirements cannot be modeled in the current representation: pools/lanes (and collaboration-level constructs) are not supported, and generation targets a single process structure. Likewise, common enterprise requirements such as task-attached boundary error/timer handling, subprocess/call-activity decomposition, and richer organizational responsibility views are outside the implemented element set. In addition, manual canvas edits are not part of the assistant’s working state unless re-imported, so follow-up AI edits may not reflect those manual changes.

5.5. Behavioral Validity and Executable Semantics

The conformance checking results presented in Table 4 provide empirical evidence that the structural improvements observed through GED and RGED metrics translate into meaningful behavioral equivalence. The average F1 scores of 0.72 for JSON versus 0.69 for XML indicate that the JSON-based approach not only produces structurally similar diagrams but also preserves the execution semantics required for workflow automation. While both approaches achieve high behavioral alignment after label normalization, the overall patterns reveal important nuances in how different models handle each representation.

The fitness dimension reveals that both JSON and XML approaches achieved comparable performance, with JSON averaging 0.84 and XML averaging 0.82, suggesting that

both representations enable LLMs to maintain valid control-flow paths during generation. However, the precision scores demonstrate more complex patterns that illuminate the relationship between model capability and representation choice. Frontier models like GPT-5.1 and Claude 4.5 Sonnet actually achieved superior precision with direct XML generation, with GPT-5.1 reaching 0.71 precision with XML compared to 0.54 with JSON, and Claude 4.5 Sonnet achieving 0.75 precision with XML versus 0.73 with JSON. This counter-intuitive finding suggests that these highly capable models possess sufficient reasoning capacity to navigate the syntactic complexity of XML while maintaining tighter semantic constraints, potentially benefiting from the explicit structural information encoded in XML tags. Conversely, smaller and open-weight models showed dramatic improvements with the JSON approach: DeepSeek V3's precision increased from 0.59 with XML to 0.76 with JSON, and Gemini 2.0 Flash improved from 0.63 to 0.77. This bifurcated pattern indicates that the JSON intermediate representation serves as an effective cognitive scaffold for models that would otherwise be overwhelmed by XML's verbosity, while frontier models may actually leverage the richer structural information present in XML to produce more precisely constrained behavior.

This observation has important implications for deployment strategies. For applications where maximum behavioral precision is the primary concern and the overhead of verbose output is acceptable, frontier models like GPT-5.1 or Claude 4.5 Sonnet may achieve optimal results with direct XML generation. However, for the majority of practical deployments, especially those constrained by cost, latency, or data privacy requirements that necessitate locally-hosted open-weight models, the JSON-based approach remains a highly practical and competitive alternative. The structured representation democratizes access to high-quality process modeling by enabling smaller models to achieve competitive semantic correctness, with models like DeepSeek V3 and Gemini 2.0 Flash actually surpassing frontier models' XML performance when using the JSON approach. This flexibility allows organizations to select the optimal representation based on their specific model choice and operational constraints.

These behavioral metrics complement the structural similarity measures and confirm that the generated models are not merely visually similar to ground truth, but semantically equivalent executable process definitions suitable for deployment in business process management systems. The conformance results further reinforce that the choice between JSON and XML representations should be informed by the specific model being deployed and the operational constraints of the organization.

5.6. Comparison with Existing Approaches

Unlike previous tools such as BPMN-Chatbot [20] and ProMoAI [21], which rely on regenerating the full process definition (via JSON or code) for every refinement, our system separates the editing logic from the model generation. While ProMoAI leverages formal methods (POWL) and BPMN-Chatbot utilizes intermediate JSON to optimize model creation, both approaches fundamentally treat modification as a conversational regeneration task. In contrast, BPMN Assistant implements a dedicated intermediate layer that supports atomic function calling, enabling targeted and deterministic updates without the overhead or instability associated with regenerating the entire process state for minor edits.

6. Limitations and Future Work

BPMN Assistant, while demonstrating promising results, has several specific limitations regarding its architecture, evaluation methodology, and scope. To provide clarity, we have grouped these limitations into key thematic areas.

Architectural and Scope Limitations: Although the system supports a robust set of elements, including intermediate events (timer, message) and inclusive gateways, the current implementation does not support collaboration diagrams (pools and lanes) or complex artifacts like data objects. This restricts the system's application in scenarios requiring multi-participant modeling.

In complex enterprise architectures, pools and lanes are frequently used to represent organizational boundaries, interacting systems, and message exchanges between participants. The inability to automatically lay out such diagrams therefore reduces end-to-end usability for modeling large cross-organizational processes in a single integrated view: even if structurally valid BPMN XML can be produced, the resulting diagram cannot be reliably visualized inside our web application without manual post-processing (e.g., arranging pools/lanes in a BPMN editor). As a practical workaround, users can decompose a collaboration model into multiple single-participant processes and iteratively refine each one; however, this shifts part of the integration effort back to the modeler.

However, as the focus of this work is on executable process orchestration, this omission does not impact the primary research objectives.

Methodological Asymmetry: A limitation of this study is the asymmetry in error handling between the two formats. The proposed system employs a JSON schema validator that triggers retries for structural inconsistencies (e.g., duplicate IDs, missing references), whereas the XML baseline relies on a single-pass generation. While this highlights the ease of implementing validation logic for structured intermediate representations, it likely contributed to the lower success rate of the XML baseline in cases where simple structural errors could have been resolved through iterative repair. Future research should investigate the efficacy of iterative repair strategies for direct XML generation to decouple the impact of the representation from the validation mechanism.

Evaluation and Dataset Constraints: While our analysis demonstrates a significant reduction in overall failure rates with the JSON approach, we did not perform a granular taxonomy of failure modes (e.g., distinguishing between invalid references versus gateway mis-joins). Our primary reliability metric focused on the binary distinction between renderable and non-renderable outputs. A more detailed qualitative analysis of the specific BPMN constructs that challenge direct XML generation remains a valuable direction for future research.

We acknowledge that our evaluation relies on synthetic process descriptions. While using real-world enterprise narratives would be ideal, a review of existing public resources reveals a significant gap in the field. Publicly available corpora, such as the Friedrich et al. [12] collection or the PET dataset [29], either lack corresponding gold-standard BPMN 2.0 models or contain diagrams that were not validated by independent experts. Furthermore, obtaining real-world process documentation is constrained by confidentiality and compliance issues. Consequently, a curated synthetic dataset was necessary to ensure the structural validity of the ground truth used for benchmarking.

Semantic and Behavioral Validation Limits: To validate the semantic correctness of generated models beyond structural similarity, we incorporate conformance checking using token-based replay on simulated event logs. This evaluation demonstrates that both JSON and XML approaches produce models with high behavioral fidelity to ground-truth processes, achieving average F1 scores of 0.72 and 0.69 respectively. While these results confirm that the generated models preserve executable semantics, it is important to note that our semantic evaluation is based on simulated traces and does not encompass the full complexity of real-world process execution, including data flow constraints, resource dependencies, and temporal specifications.

The evaluation does not provide exhaustive formal verification beyond replay-based assessment, such as soundness or deadlock-freedom checks, nor does it evaluate conformance against real execution data from production systems. Additionally, we do not provide comprehensive performance analysis through large-scale simulation studies or full usability evaluation with end users.

Usability and Model Dependencies: The performance of BPMN Assistant is heavily dependent on the underlying LLM. Our evaluation revealed significant variations in performance across different models, with Claude 4.5 Sonnet and GPT-5.1 showing particularly strong results. Further investigation into the specific capabilities and limitations of different LLMs in the context of process modeling would provide valuable insights for future improvements.

Human-computer interaction (HCI) studies would be necessary to properly evaluate the usability of the system for non-technical users. While BPMN Assistant aims to lower the barrier to process modeling through natural language interaction, comprehensive user studies would be required to validate this claim.

Another area of concern is the reliance on clear and unambiguous natural language input. Users might encounter difficulties in achieving this clarity, especially in multilingual or context-specific scenarios where language and terminology nuances can introduce ambiguity.

The evaluation approach used in this research was selected considering the research objectives, the method of model generation, and existing evaluation methods cited in the literature. Future work could address these limitations by extending the range of supported BPMN elements, broadening semantic validation with additional formal methods and real-world event logs, conducting usability studies, and optimizing the system for specific LLMs based on their strengths in process modeling tasks.

7. Conclusions

This paper presented BPMN Assistant, a novel approach to business process modeling that leverages LLMs to bridge the gap between natural language descriptions and formal BPMN representations. Through a comprehensive evaluation using state-of-the-art models, including GPT-5.1, Claude 4.5 Sonnet, and DeepSeek V3, we demonstrated that using a simplified JSON structure instead of raw XML yields measurable improvements in reliability and efficiency.

Our results indicate that while direct XML generation is feasible for creating simple models from scratch, it is often insufficient for iterative modification tasks. The proposed JSON-based approach outperformed or matched direct XML manipulation across all tested models in editing scenarios. While frontier models like Claude 4.5 Sonnet achieved parity in both modalities, the JSON advantage was critical for other models. This was most pronounced in open-weight models, where the structured approach enabled DeepSeek V3 to achieve a viable 50% success rate compared to a near-total failure (8%) with standard XML prompting. This suggests that intermediate representations are a key enabler for deploying cost-effective, locally hosted models in process automation workflows.

Conformance checking analysis validates that the generated models maintain semantic correctness, with both approaches achieving high behavioral fidelity to ground-truth processes and F1 scores exceeding 0.69. While frontier models like GPT-5.1 and Claude 4.5 Sonnet can achieve high precision with either representation, the JSON-based approach proved critical for democratizing access to process modeling by enabling smaller and open-weight models to achieve competitive semantic correctness. This is particularly significant for organizations constrained by cost, latency, or data privacy requirements that necessitate local model deployment, where the JSON approach enables models like

DeepSeek V3 and Gemini 2.0 Flash to surpass even frontier models' XML performance in behavioral precision.

In terms of performance, the trade-off between increased input context and reduced output complexity proved highly advantageous. By shifting the computational burden to the input context, our approach reduced editing latency by approximately 43% and decreased output token usage by over 75%. These efficiency gains are critical for the practical adoption of LLM-based tools in interactive, real-time environments.

Looking ahead, future work will focus on conducting comprehensive usability studies to validate the system's effectiveness with non-technical business users. Additionally, we aim to further optimize the JSON schema to better handle complex cyclic dependencies and improve the system's robustness when handling highly ambiguous natural language descriptions.

Author Contributions: Conceptualization, J.T.L. and N.T.; methodology, J.T.L. and N.T.; software, J.T.L.; validation, J.T.L. and N.T.; investigation, J.T.L.; resources, N.T. and D.E.; data curation, J.T.L.; writing—original draft preparation, J.T.L.; writing—review and editing, N.T. and D.E.; visualization, J.T.L.; supervision, N.T. and D.E.; project administration, N.T.; Funding acquisition, N.T. All authors have read and agreed to the published version of the manuscript.

Funding: This research was supported by the project *GENESIS—Generative Artificial Intelligence in Modeling and Execution of Distributed Enterprise Software Information Systems* (CroRIS ID: 15519, project code IIP_010136), as well as EC Digital Europe Programme EDIH Adria 2.0 (101256325); SPIN projects IP.1.1.03.0120, IP.1.1.03.0028 and IP.1.1.03.0039; and the Juraj Dobrila University of Pula institutional research project IIP_010144.

Institutional Review Board Statement: Not applicable.

Informed Consent Statement: Not applicable.

Data Availability Statement: The system source code is available at <https://github.com/jtlicardo/bpmn-assistant> (accessed on 21 February 2026). The evaluation dataset is available at <https://huggingface.co/datasets/jtlicardo/bpmn-assistant-eval> (accessed on 21 February 2026). The custom evaluation metrics code is available at <https://github.com/jtlicardo/bpmn-ged> (accessed on 21 February 2026).

Acknowledgments: During the preparation of this manuscript, the authors used Gemini 3 Pro to improve language and readability. The authors have reviewed and edited the output and take full responsibility for the content of this publication.

Conflicts of Interest: The authors declare no conflicts of interest.

Abbreviations

The following abbreviations are used in this manuscript:

BPMN	Business Process Model and Notation
LLM	Large Language Model
JSON	JavaScript Object Notation
XML	Extensible Markup Language
BPM	Business Process Management
GED	Graph Edit Distance
RGED	Relative Graph Edit Distance
AI	Artificial Intelligence
API	Application Programming Interface
GUI	Graphical User Interface
HCI	Human-Computer Interaction

Appendix A. Process Example

The following examples demonstrate how the JSON intermediate representation handles complex BPMN constructs, including cyclic flows (loops) and event definitions.

Appendix A.1. Cyclic Flow (Loops)

This example demonstrates the use of the `next` attribute to model a loop where a student must retake an exam upon failure. This illustrates the system's ability to handle non-hierarchical flows.

Textual description:

"Someone starts a process by entering an exam room. After that, they take the test. Once the exam is finished, their score is checked. If they scored more than 50%, their grade is recorded and the process ends. But if they fail, they have to go back to the beginning and take the exam again."

JSON Representation:

```
{
  "process": [
    {
      "type": "startEvent",
      "id": "start"
    },
    {
      "type": "task",
      "id": "task1",
      "label": "Enter exam room"
    },
    {
      "type": "task",
      "id": "task2",
      "label": "Take test"
    },
    {
      "type": "exclusiveGateway",
      "id": "eg",
      "label": "Score more than 50%?",
      "has_join": false,
      "branches": [
        {
          "condition": "Yes",
          "path": [
            {
              "type": "task",
              "id": "task3",
              "label": "Enter grade"
            },
            {
              "type": "endEvent",
              "id": "end1"
            }
          ]
        }
      ]
    }
  ]
}
```

```

        },
        {
            "condition": "No",
            "path": [],
            "next": "task1" // Redirects flow back to the first task
        }
    ]
}
]
}

```

Appendix A.2. Event-Driven Workflow

This example demonstrates the support for specific event definitions (Timer and Message events) required for executable process models.

Textual description:

“A batch process starts at midnight using a timer. The system processes pending orders and waits for 5 min. Then it sends a completion notification and waits to receive a confirmation message before ending.”

JSON Representation:

```

{
  "process": [
    {
      "type": "startEvent",
      "id": "start",
      "label": "Start at midnight",
      "eventDefinition": "timerEventDefinition"
    },
    {
      "type": "serviceTask",
      "id": "task1",
      "label": "Process pending orders"
    },
    {
      "type": "intermediateCatchEvent",
      "id": "timer1",
      "label": "Wait 5 minutes",
      "eventDefinition": "timerEventDefinition"
    },
    {
      "type": "intermediateThrowEvent",
      "id": "message1",
      "label": "Send completion notification",
      "eventDefinition": "messageEventDefinition"
    },
    {
      "type": "intermediateCatchEvent",
      "id": "message2",
      "label": "Wait for confirmation",
      "eventDefinition": "messageEventDefinition"
    },
  ],
}

```

```

    {
      "type": "endEvent",
      "id": "end"
    }
  ]
}

```

Appendix B. Prompts Used in the Evaluation

This appendix provides the exact prompt templates used in our experiments.

Appendix B.1. XML Baseline Prompt

System prompt (verbatim):

You are a BPMN process modeling expert. Create valid BPMN XML without DI
 → information. Output only the BPMN XML, no comments or other text.

Only use these BPMN elements:

- task
- userTask
- serviceTask
- exclusiveGateway
- parallelGateway
- startEvent
- endEvent

Ensure each element has:

- A unique id attribute
- Incoming and outgoing sequence flows defined within the element
- Corresponding sequenceFlow elements with sourceRef and targetRef attributes

Example structure:

```

<bpmn:task id="Activity_1" name="Task">
  <bpmn:incoming>Flow_1</bpmn:incoming>
  <bpmn:outgoing>Flow_2</bpmn:outgoing>
</bpmn:task>
<bpmn:sequenceFlow id="Flow_1" sourceRef="StartEvent_1" targetRef="Activity_1" />

```

Do not include:

- pools
- lanes
- participants
- any other BPMN elements
- DI information

The output should be valid BPMN XML starting with:

```

<?xml version="1.0" encoding="UTF-8"?>
<bpmn:definitions xmlns:bpmn="http://www.omg.org/spec/BPMN/20100524/MODEL" ...>
  <bpmn:process id="Process_1" isExecutable="true">
    ... your elements here ...
  </bpmn:process>
</bpmn:definitions>

```

Appendix B.2. JSON Prompt Templates

BPMN Assistant uses multiple prompt templates for the JSON-based approach (e.g., initial generation, atomic edit-function calling, and error-correction after validation failures). The corresponding templates are available in the public BPMN Assistant repository referenced in the Introduction.

Appendix B.3. Semantic Label Normalization Prompt

Prompt (verbatim):

The model was invoked using the default decoding configuration (temperature = 1.0), which is fixed by the provider for this model family and cannot be modified.

```
PROMPT = """Normalize BPMN task, event, and sequence flow labels by mapping them
→ to simple letter names (A, B, C, etc.):
- Match semantically similar elements (tasks, events, flows)
- Use ONLY letters (A, B, C, etc.) as normalized names for ALL elements
- Same elements get same letters ONLY if they are semantically similar
- Different/unrelated elements MUST get different letters
- Consider element sequence via edges
- Only normalize elements that have labels/names - ignore unlabeled elements
- Do not normalize IDs, only normalize labels/names
```

Examples:

Input:

```
{
  "g1": [
    {"name": "Submit order", "type": "task", "id": "1"},
    {"name": "Process payment", "type": "task", "id": "2"}
  ],
  "g2": [
    {"name": "Send order", "type": "task", "id": "task_1"},
    {"name": "Handle payment", "type": "task", "id": "task_2"}
  ],
  "e1": [
    {"name": "Order submitted", "source": "1", "target": "2"}
  ],
  "e2": [
    {"name": "Order sent", "source": "task_1", "target": "task_2"}
  ]
}
```

Expected Output:

```
{
  "name_mappings": [
    {"original_name": "Submit order", "normalized_name": "A"},
    {"original_name": "Send order", "normalized_name": "A"},
    {"original_name": "Process payment", "normalized_name": "B"},
    {"original_name": "Handle payment", "normalized_name": "B"},
    {"original_name": "Order submitted", "normalized_name": "C"},
    {"original_name": "Order sent", "normalized_name": "C"}
  ]
}
```

Appendix C. Cost Model Breakdown

For readability, the detailed per-generation cost breakdown table is moved here from the main text.

Table A1. Estimated token cost per generation (USD cents), broken down into input, cached input, and output costs, plus totals under two assumptions: (i) no caching (all input billed at standard input rate) and (ii) warm cache (all input billed at cached input rate).

Model	Modality	Input (c)	Input Cached (c)	Output (c)	Total No Cache (c)	Total Cached (c)
GPT-4o	JSON	0.621962	0.310981	0.483070	1.105033	0.794051
GPT-4o	XML	0.109420	0.054710	1.348970	1.458390	1.403680
GPT-5.1	JSON	0.321710	0.032171	0.568120	0.889830	0.600291
GPT-5.1	XML	0.056210	0.005621	1.602030	1.658240	1.607651
GPT-5 mini	JSON	0.064504	0.006450	0.177720	0.242224	0.184170
GPT-5 mini	XML	0.011242	0.001124	0.323844	0.335086	0.324968
Claude 4.5 Sonnet	JSON	0.926970	0.092697	0.926175	1.853145	1.018872
Claude 4.5 Sonnet	XML	0.156060	0.015606	2.749950	2.906010	2.765556
Claude 3.5 Sonnet	JSON	0.879960	0.087996	0.911220	1.791180	0.999216
Claude 3.5 Sonnet	XML	0.155760	0.015576	2.415825	2.571585	2.431401
Gemini 2.0 Flash	JSON	0.027952	0.006988	0.022240	0.050192	0.029228
Gemini 2.0 Flash	XML	0.004652	0.001163	0.061724	0.066376	0.062887
Llama 3.3 70B Instruct	JSON	0.226400	0.113200	0.031683	0.258083	0.144883
Llama 3.3 70B Instruct	XML	0.043679	0.021839	0.136182	0.179861	0.158021
Qwen 2.5 72B Instruct	JSON	0.235790	0.117895	0.035317	0.271107	0.153212
Qwen 2.5 72B Instruct	XML	0.042149	0.021074	0.137016	0.179165	0.158090
DeepSeek V3	JSON	0.140685	0.070342	0.066237	0.206922	0.136580
DeepSeek V3	XML	0.026389	0.013194	0.247094	0.273483	0.260289

References

- Object Management Group. *Business Process Model and Notation (BPMN), Version 2.0*; Technical report; Object Management Group (OMG): Milford, MA, USA, 2010.
- Rosemann, M. Potential Pitfalls of Process Modeling: Part A. *Bus. Process Manag. J.* **2006**, *12*, 377–384. [[CrossRef](#)]
- Recker, J. Opportunities and Constraints: The Current Struggle with BPMN. *Bus. Process Manag. J.* **2010**, *16*, 181–201. [[CrossRef](#)]
- Njanka, S.Q.; Sandula, G.; Colomo-Palacios, R. IT-Business Alignment: A Systematic Literature Review. *Procedia Comput. Sci.* **2021**, *181*, 333–340. [[CrossRef](#)]
- van der Aa, H.; Leopold, H.; Mannhardt, F.; Reijers, H.A. On the Fragmentation of Process Information: Challenges, Solutions, and Outlook. In *Proceedings of the Enterprise, Business-Process and Information Systems Modeling*; Gaaloul, K., Schmidt, R., Nurcan, S., Guerreiro, S., Ma, Q., Eds.; Springer: Cham, Switzerland, 2015; pp. 3–18. [[CrossRef](#)]
- Bellan, P.; Dragoni, M.; Ghidini, C. A Qualitative Analysis of the State of the Art in Process Extraction from Text. In *Proceedings of the AIXIA 2020 Discussion Papers Workshop Co-Located with the 19th International Conference of the Italian Association for Artificial Intelligence (AIXIA2020)*, Online, 27 November 2020.
- Goncalves, J.C.d.A.R.; Santoro, F.M.; Baião, F.A. Let Me Tell You a Story—On How to Build Process Models. *J. Univers. Comput. Sci.* **2011**, *17*, 276–295. [[CrossRef](#)]
- Weber, B.; Neurauter, M.; Pinggera, J.; Zugall, S.; Furtner, M.; Martini, M.; Sachse, P. Measuring Cognitive Load During Process Model Creation. In *Proceedings of the Information Systems and Neuroscience*; Davis, F.D., Riedl, R., vom Brocke, J., Léger, P.M., Randolph, A.B., Eds.; Springer: Cham, Switzerland, 2015; pp. 129–136. [[CrossRef](#)]
- Desalegn, E.G.; Guedes, M.J.C.; Da Silva Gomes, J.F.; Tebeka, S.M. Disentangling Organizational Agility from Flexibility, Adaptability, and Versatility: A Systematic Review. *Future Bus. J.* **2024**, *10*, 117. [[CrossRef](#)]
- Leopold, H.; Mendling, J.; Günther, O. Learning from Quality Issues of BPMN Models from Industry. *IEEE Softw.* **2016**, *33*, 26–33. [[CrossRef](#)]
- Van Der Aalst, W.; Weijters, A. Process Mining: A Research Agenda. *Comput. Ind.* **2004**, *53*, 231–244. [[CrossRef](#)]
- Friedrich, F.; Mendling, J.; Puhlmann, F. Process Model Generation from Natural Language Text. In *Proceedings of the Advanced Information Systems Engineering*; Mouratidis, H., Rolland, C., Eds.; Springer: Berlin/Heidelberg, Germany, 2011; pp. 482–496. [[CrossRef](#)]
- Leopold, H.; Mendling, J.; Polyvyanyy, A. Generating Natural Language Texts from Business Process Models. In *Proceedings of the Advanced Information Systems Engineering*; Ralyté, J., Franch, X., Brinkkemper, S., Wrycza, S., Eds.; Springer: Berlin/Heidelberg, Germany, 2012; pp. 64–79. [[CrossRef](#)]
- Mendling, J.; Leopold, H.; Thom, L.H. Natural Language Processing with Process Models (NLP4RE Report Paper). In *Proceedings of the CEUR Workshop Proceedings*; RWTH Aachen: Aachen, Germany, 2019.

15. Grohs, M.; Abb, L.; Elsayed, N.; Rehse, J.R. Large Language Models Can Accomplish Business Process Management Tasks. In *Proceedings of the Business Process Management Workshops*; De Weerd, J., Pufahl, L., Eds.; Springer: Cham, Switzerland, 2024; pp. 453–465. [\[CrossRef\]](#)
16. Kampik, T.; Warmuth, C.; Rebmann, A.; Agam, R.; Egger, L.N.P.; Gerber, A.; Hoffart, J.; Kolk, J.; Herzig, P.; Decker, G.; et al. Large Process Models: Business Process Management in the Age of Generative AI. *arXiv* **2023**, arXiv:2309.00900. [\[CrossRef\]](#)
17. Linkon, A.A.; Shaima, M.; Sarker, M.S.U.; Badruddowza; Nabi, N.; Rana, M.N.U.; Ghosh, S.K.; Rahman, M.A.; Esa, H.; Chowdhury, F.R. Advancements and Applications of Generative Artificial Intelligence and Large Language Models on Business Management: A Comprehensive Review. *J. Comput. Sci. Technol. Stud.* **2024**, *6*, 225–232. [\[CrossRef\]](#)
18. Klievtsova, N.; Benzin, J.V.; Kampik, T.; Mangler, J.; Rinderle-Ma, S. Conversational Process Modeling: Can Generative AI Empower Domain Experts in Creating and Redesigning Process Models? *arXiv* **2023**, arXiv:2304.11065. [\[CrossRef\]](#)
19. Rebmann, A.; Schmidt, F.D.; Glavaš, G.; Van Der Aa, H. Evaluating the Ability of LLMs to Solve Semantics-Aware Process Mining Tasks. In *2024 6th International Conference on Process Mining (ICPM)*; IEEE: New York, NY, USA, 2024; pp. 9–16. [\[CrossRef\]](#)
20. Köpke, J.; Safan, A. Introducing the BPMN-Chatbot for Efficient LLM-Based Process Modeling. In *Proceedings of the International Conference on Business Process Management*, Krakow, Poland, 1–6 September 2024.
21. Kourani, H.; Berti, A.; Schuster, D.; Van Der Aalst, W.M. ProMoAI: Process Modeling with Generative AI. In *Proceedings of the Thirty-Third International Joint Conference on Artificial Intelligence*, Jeju, Republic of Korea, 3–9 August 2024; pp. 8708–8712. [\[CrossRef\]](#)
22. Berti, A.; Qafari, M.S. Leveraging Large Language Models (LLMs) for Process Mining (Technical Report). *arXiv* **2023**, arXiv:2307.12701. [\[CrossRef\]](#)
23. Kourani, H.; Berti, A.; Schuster, D.; van der Aalst, W.M.P. Process Modeling With Large Language Models. In *Enterprise, Business-Process and Information Systems Modeling*; Springer: Cham, Switzerland, 2024; Volume 511, pp. 229–244. [\[CrossRef\]](#)
24. Ferreira, R.C.B.; Thom, L.H.; Fantinato, M. A Semi-automatic Approach to Identify Business Process Elements in Natural Language Texts. In *Proceedings of the 19th International Conference on Enterprise Information Systems*; SciTePress: Setúbal, Portugal, 2017; pp. 250–261. [\[CrossRef\]](#)
25. Neuberger, J.; Ackermann, L.; van der Aa, H.; Jablonski, S. A Universal Prompting Strategy for Extracting Process Model Information from Natural Language Text Using Large Language Models. In *Proceedings of the Conceptual Modeling—43rd International Conference, ER 2024, Pittsburgh, PA, USA, 28–31 October 2024*; Lecture Notes in Computer Science; Springer Nature: Cham, Switzerland, 2024; Volume 15238, pp. 38–55. [\[CrossRef\]](#)
26. Busch, K.; Rochlitz, A.; Sola, D.; Leopold, H. Just Tell Me: Prompt Engineering in Business Process Management. *arXiv* **2023**, arXiv:2304.07183. [\[CrossRef\]](#)
27. Kourani, H.; Berti, A.; Schuster, D.; van der Aalst, W.M.P. Evaluating Large Language Models on Business Process Modeling: Framework, Benchmark, and Self-Improvement Analysis. *arXiv* **2024**, arXiv:2412.00023. [\[CrossRef\]](#)
28. Hörner, L.F.; Möller, M.; Reichert, M. Automatically Generating BPMN 2.0 Process Models from Natural Language Process Descriptions: Challenges, Framework, Quality Assessment. *Bus. Inf. Syst. Eng.* **2026**. [\[CrossRef\]](#)
29. Bellan, P.; Dragoni, M.; Ghidini, C.; van der Aa, H.; Ponzetto, S.P. Process Extraction from Text: Benchmarking the State of the Art and Paving the Way for Future Challenges. *arXiv* **2023**, arXiv:2110.03754. [\[CrossRef\]](#)
30. Nivon, Q.; Salaün, G. Automated Generation of BPMN Processes from Textual Requirements. In *Proceedings of the Service-Oriented Computing*; Gaaloul, W., Sheng, M., Yu, Q., Yangui, S., Eds.; Springer: Singapore, 2025; pp. 185–201. [\[CrossRef\]](#)
31. OpenAI; Hurst, A.; Lerer, A.; Goucher, A.P.; Perelman, A.; Ramesh, A.; Clark, A.; Ostrow, A.J.; Welihinda, A.; Hayes, A.; et al. GPT-4o System Card. *arXiv* **2024**, arXiv:2410.21276. [\[CrossRef\]](#)
32. Comanici, G.; Bieber, E.; Schaekermann, M.; Pasupat, I.; Sachdeva, N.; Dhillon, I.; Blistein, M.; Ram, O.; Zhang, D.; Rosen, E.; et al. Gemini 2.5: Pushing the Frontier with Advanced Reasoning, Multimodality, Long Context, and Next Generation Agentic Capabilities. *arXiv* **2025**, arXiv:2507.06261. [\[CrossRef\]](#)
33. Grattafiori, A.; Dubey, A.; Jauhri, A.; Pandey, A.; Kadian, A.; Al-Dahle, A.; Letman, A.; Mathur, A.; Schelten, A.; Vaughan, A.; et al. The Llama 3 Herd of Models. *arXiv* **2024**, arXiv:2407.21783. [\[CrossRef\]](#)
34. Qwen; Yang, A.; Yang, B.; Zhang, B.; Hui, B.; Zheng, B.; Yu, B.; Li, C.; Liu, D.; Huang, F.; et al. Qwen2.5 Technical Report. *arXiv* **2025**, arXiv:2412.15115. [\[CrossRef\]](#)
35. DeepSeek-AI; Liu, A.; Feng, B.; Xue, B.; Wang, B.; Wu, B.; Lu, C.; Zhao, C.; Deng, C.; Zhang, C.; et al. DeepSeek-V3 Technical Report. *arXiv* **2024**, arXiv:2412.19437. [\[CrossRef\]](#)
36. Licardo, J.T.; Tanković, N.; Etinger, D. A Method for Extracting BPMN Models from Textual Descriptions Using Natural Language Processing. *Procedia Comput. Sci.* **2024**, *239*, 483–490. [\[CrossRef\]](#)
37. Dijkman, R.; Dumas, M.; García-Bañuelos, L. Graph Matching Algorithms for Business Process Model Similarity Search. In *Proceedings of the Business Process Management*; Dayal, U., Eder, J., Koehler, J., Reijers, H.A., Eds.; Springer: Berlin/Heidelberg, Germany, 2009; Volume 5701, pp. 48–63. [\[CrossRef\]](#)

38. Sohail, A.; Haseeb, A.; Rehman, M.; Durai Dominic, D.; Arif Butt, M. An Intelligent Graph Edit Distance-Based Approach for Finding Business Process Similarities. *Comput. Mater. Contin.* **2021**, *69*, 3603–3618. [[CrossRef](#)]
39. Waspada, I.; Sarno, R. An Improved Method of Graph Edit Distance for Business Process Model Similarity Measurement. In *2020 4th International Conference on Informatics and Computational Sciences (ICICoS)*; IEEE: New York, NY, USA, 2020; pp. 1–6. [[CrossRef](#)]
40. Schoknecht, A.; Thaler, T.; Fettke, P.; Oberweis, A.; Laue, R. Similarity of Business Process Models—A State-of-the-Art Analysis. *ACM Comput. Surv.* **2018**, *50*, 52. [[CrossRef](#)]
41. Huang, Z.; Kumar, A. New Quality Metrics for Evaluating Process Models. In *Business Process Management Workshops*; Ardagna, D., Mecella, M., Yang, J., Eds.; Springer: Berlin/Heidelberg, Germany, 2009; Volume 17, pp. 164–170. [[CrossRef](#)]
42. Pavlicek, J.; Hronza, R.; Pavlickova, P.; Jelinkova, K. The Business Process Model Quality Metrics. In *Proceedings of the Enterprise and Organizational Modeling and Simulation*; Pergl, R., Lock, R., Babkin, E., Molhanec, M., Eds.; Springer: Cham, Switzerland, 2017; Volume 298, pp. 134–148. [[CrossRef](#)]
43. Berti, A.; van Zelst, S.; Schuster, D. PM4Py: A Process Mining Library for Python. *Softw. Impacts* **2023**, *17*, 100556. [[CrossRef](#)]

Disclaimer/Publisher’s Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.